Journal of King Saud University Computer and Information Sciences (2026)38:918
https://doi.org/10.1007/s44443-026-01332-y

RESEARCH

A lightweight distributed intrusion detection framework for SDN via unsupervised feature augmentation and collaborative LightGBM

Yufeng Jia[1,2] · Jiadong Ren[2] · Haitao He[2] · Xianshan Li[2] · Ke Xu[2]

Received: 17 June 2026 / Accepted: 31 August 2026
© The Author(s) 2026

Abstract

With the rapid advancement of Internet technologies, the Internet of Things (IoT), edge computing, and software-defined networking (SDN) have emerged as key technologies for large-scale data processing in resource-constrained environments. However, the centralized control paradigm of SDN places substantial processing demands on a single controller, potentially resulting in excessive bandwidth overhead, high-dimensional data-processing challenges, and low computational efficiency. Moreover, the continuous data analytics and real-time policy enforcement required in heterogeneous networks further exacerbate controller-side performance bottlenecks. To address this issue, this study proposes a lightweight intrusion detection framework based on unsupervised representation learning and distributed collaborative learning, providing a potential perspective for achieving efficient classification in resource-constrained SDN environments. The principal contributions of this study are summarized as follows. (i) Unsupervised representation learning is employed to enrich the original feature space, while three complementary feature-selection strategies are used to identify the most informative features. This process preserves essential information while substantially reducing computational complexity. (ii) From a system-design perspective, the proposed framework exploits SDN programmability and the local processing capabilities of network nodes to distribute training tasks across individual nodes. This localized training architecture accelerates model training while effectively reducing communication overhead. (iii) Using only 13 key features selected from the original 83-dimensional feature set, the proposed method achieves an F1-score of 98.76%, with training times ranging from 9.65 to 13.65 seconds. These results demonstrate that the framework effectively balances detection performance and computational efficiency in distributed SDN environments. Overall, this study presents a lightweight distributed intrusion detection solution tailored to resource-constrained SDN environments.

Keywords Software-Defined networking · Intrusion detection · Distributed · Unsupervised representation learning

1 Introduction

Software-Defined Networking (SDN) decouples the control plane from the data plane, enabling centralized network control, programmability, and dynamic resource scheduling. These capabilities significantly enhance network flexibility and management efficiency, making SDN a foundational technology for large-scale data center networks, cloud platforms, and 5G core infrastructures (Chen et al. 2025). However, with the rapid expansion of the Internet of Things and the proliferation of heterogeneous devices and increasingly complex computational tasks, the centralized controller architecture of SDN introduces new security challenges. These include single points of failure, DDoS/DoS attacks, flow table overflows, link saturation, and vulnerabilities in southbound and northbound interfaces (Kaur et al. 2025). Traditional intrusion detection methods based on rules or signature matching struggle to adapt to the dynamic and programmable nature of network traffic. In contrast, machine learning (ML) and deep learning (DL) approaches, due to their ability to capture

✉ Yufeng Jia
jiayufeng04@126.com

✉ Jiadong Ren
jdren@ysu.edu.cn

[1] School of Information Science and Engineering, Xinjiang College of Science and Technology, Korla, China

[2] School of Artificial Intelligence(School of Software), Yanshan University, Qinhuangdao, China

Published online: 19 October 2026

complex patterns, have become the dominant solutions in SDN-based intrusion detection (Nunez-Agurto et al. 2023).

First, intrusion detection solutions can be categorized into three levels according to their effectiveness: (i) those that can effectively handle known threats; (ii) those that can identify known threats but fail to effectively mitigate them; and (iii) those that are unable to detect or respond to unknown threats. Representation learning can enhance the model's ability to characterize latent anomalous patterns and potentially improve its generalization to unseen attacks, thus emerging as a promising direction in intrusion detection research (Jiang et al. 2026; Xi et al. 2026; Zhang et al. 2026). The underlying motivation is that raw network traffic features (e.g., packet counts, byte volumes, durations, and protocol flags) often encode anomalous patterns in subtle and nonlinear ways, making them difficult for supervised classifiers to directly exploit. Although unsupervised models can generate enriched feature representations, the resulting feature space may still contain redundant or noisy dimensions. In this context, feature selection provides a complementary inductive bias, guiding subsequent supervised learning toward more expressive and discriminative decision boundaries. For instance, Sattarpour et al. proposed an efficient BERT-based intrusion detection system (EBIDS), which incorporates feature extraction during the data preprocessing stage to reduce the complexity of high-dimensional datasets and improve model efficiency (Sattarpour et al. 2024). Although this feature extraction strategy improves processing efficiency in resource-constrained environments by avoiding the direct use of numerous redundant transformed features, it has yet to achieve favorable trade-off among detection performance, computational overhead, and deployment feasibility.

Second, in practical deployment scenarios, conventional ML/DL-based intrusion detection methods typically rely on a centralized server to perform iterative training on entire datasets. This paradigm increases detection latency, reduces computational efficiency, and incurs substantial deployment costs, particularly in resource-constrained environments (Xie et al. 2025). Existing distributed solutions attempt to mitigate these issues. Federated learning schemes enable collaborative training without raw data exchange, but impose substantial communication overhead on SDN southbound interfaces (Olanrewaju-George and Pranggono 2025). Edge-assisted distributed detection architectures exploit topology or edge-computing priors to detect stealthy attacks, yet remain hindered by computationally intensive deep neural networks unsuitable for resource-constrained switches (Alotaibi 2025; Bhoi et al. 2025). Furthermore, with respect to feature processing, most existing distributed approaches rely directly on raw traffic features and devote limited attention to learning deeper data representations that reflect the intrinsic characteristics of network traffic and the constraints imposed by practical deployment environments (Ali 2025).

To overcome these limitations, unsupervised representation learning enables the automatic extraction of more discriminative feature representations from high-dimensional raw network traffic, which not only reduces the dependency on labeled data but also improves the model's ability to capture underlying anomalous patterns. By further incorporating distributed collaboration and ensemble learning, one can exploit the inherent characteristics of SDN architectures for load-balanced training and reduced communication overhead.

Against this background, this study builds on XGBOD (Zhao and Hryniewicki 2018) by adopting its core idea of generating auxiliary features through unsupervised models, while extending it to multiclass SDN intrusion detection through a redesigned feature-generation strategy and LightGBM-based distributed classification. The main contributions are as follows:

1. To enhance the capability of supervised models to characterize latent anomalous patterns, this study leverages unsupervised representation learning to derive more informative latent representations from raw network traffic. Building on these representations, a multi-strategy feature selection mechanism is employed by integrating precise, balanced, and random selection strategies. By combining these three complementary strategies, the mechanism seeks to improve detection performance while preserving essential information and reducing feature redundancy. In doing so, it helps mitigate the additional computational overhead introduced by representation learning, and strikes a practical balance between detection accuracy and computational cost.
2. The framework is instantiated in a distributed manner by leveraging LightGBM's native parallel and distributed training support. This allows multiple SDN nodes to collaboratively train models using only locally available traffic data, thus avoiding centralized data aggregation and helping to mitigate the controller's processing bottleneck in resource-constrained environments. In doing so, the design seeks a practical balance among detection accuracy, computational overhead, and inference latency for distributed SDN deployments, offering a potentially lightweight and efficient distributed detection approach for resource-constrained SDN environments.

To account for both the networking characteristics of SDN and practical engineering requirements, this study

adopts an application-oriented perspective and systematically adapts and refines a supervised-learning approach enhanced by representation learning, with the aim of improving its suitability for resource-constrained and distributed SDN environments. In practical distributed settings, the resulting framework seeks to establish a reasonable trade-off among detection accuracy, computational complexity, and inference latency.

2 The unsupervised augmentation and distributed collaborative framework

SDN addresses the complexity and coupling issues of traditional networks by decoupling the control plane from the data plane. The control plane is responsible for decision-making, while the data plane is responsible for forwarding data packets. As shown in Fig. 1, the overall architecture of the SDN defense system, the modules collaborate closely through layering and interfaces, enabling the detection of attack traffic and the dynamic update of flow table rules. The data plane consists of SDN switches, which collect switch status and traffic statistics (e.g., packet rate, traffic volume) through the traffic collection module. This information is reported to the control plane via the OpenFlow protocol, specifically through the OFPFlowStatsRequest and OFPFlowStatsReply messages. The RYU controller receives and forwards this data to the application plane, where the unsupervised enhancement and distributed collaboration model deployed on it performs feature extraction and classification, providing decision-making input to the control plane. When abnormal traffic is detected, the application plane sends rule update instructions to the control plane via the northbound interface. The control plane generates new flow table rules via the flow table update module and sends them to the SDN switches in the data plane through the OFPFlowMod message. The new rules are immediately deployed on the switches, enabling real-time blocking of abnormal traffic while ensuring continuous forwarding of normal traffic, thus maintaining network stability. This layered architecture establishes a closed-loop network security defense mechanism, covering data collection, attack detection, and policy deployment.

It should be emphasized that Fig. 1 is intended to illustrate the placement of the proposed model within the SDN architecture and its interactions with the different architectural planes. This study focuses on adapting and refining the representation-learning-enhanced supervised-learning approach to improve its suitability for resource-constrained and distributed SDN environments. Lower-level operations, such as data collection and flow-rule installation, follow the scheme described in Jia et al. (2026), whereas end-to-end latency measurement and latency-

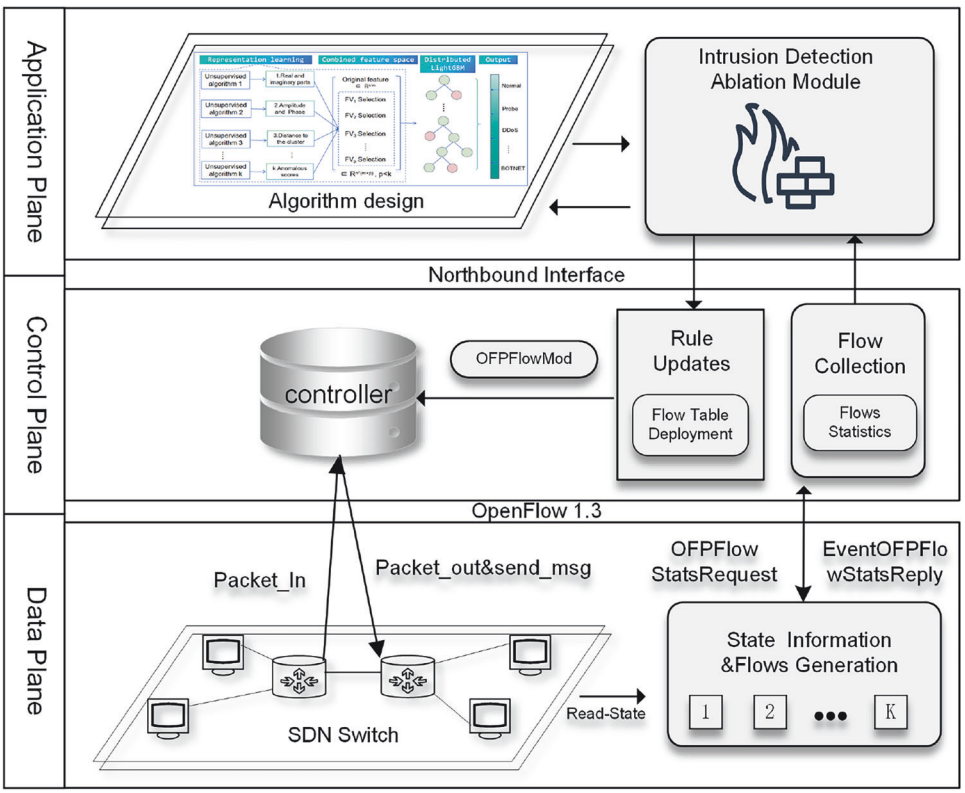


Fig. 1 Overall Architecture of the SDN Defense System

control strategies may be implemented using the methods presented in Jia et al. (2025). Owing to space limitations, detailed descriptions of these supporting modules and the overall system framework are omitted here; further discussion is available in the authors' related work for interested readers.

2.1 Algorithm framework

The effectiveness of machine learning models is highly dependent on the quality of data representations. However, manual annotation is costly, and supervised methods often struggle to capture complex patterns in highly imbalanced datasets. Consequently, increasing attention has been directed toward unsupervised approaches that extract richer feature representations from limited unlabeled data. Unsupervised anomaly detection aims to identify observations that deviate significantly from expected behavioral patterns without relying extensively on labeled samples. Chandola et al. (2009) provided a comprehensive taxonomy of anomaly detection techniques and highlighted the critical role of feature representation quality in distinguishing anomalous behavior from normal patterns. In network intrusion detection, effective feature engineering and feature selection are equally essential, as raw network traffic data often contain redundant, noisy, or weakly discriminative attributes. Constructing compact yet information-rich feature representations can therefore improve detection accuracy, computational efficiency, and model generalization.

Motivated by these observations, numerous unsupervised representation learning methods have been used to enhance feature expressiveness and subsequently improve the performance of supervised learning models. These approaches include density- and distance-based methods (e.g., K-Means and DBSCAN), statistical and probabilistic models (e.g., Gaussian Mixture Models (GMM) and Probabilistic Principal Component Analysis (PPCA)), spectral methods (e.g., wavelet transforms), matrix factorization and linear techniques (e.g., PCA, NMF, and SVD), deep representation learning methods (e.g., Variational Autoencoders (VAE) and contrastive learning), as well as graph embedding and sparse representation techniques (Fernandes et al. 2019; Li et al. 2024). Each method exhibits distinct advantages and limitations under different application scenarios. When appropriately combined, they can effectively enrich the original feature space and generate more expressive intermediate or final feature vectors (FVs). Compared with conventional supervised approaches, unsupervised anomaly detection methods are more capable of capturing complex patterns and underlying data distributions in highly imbalanced datasets. Consequently, incorporating their outputs as complementary inputs to supervised classifiers provides an effective strategy for feature representation learning.

Building upon these principles, this study draws on the core idea of XGBOD proposed by Yue Zhao et al. (2018), namely, enhancing the original feature space with auxiliary representations generated by unsupervised detection models. However, our method is not a direct implementation of the original XGBOD framework. While XGBOD was originally developed for binary anomaly detection, the proposed method adapts this feature-enhancement paradigm to multiclass SDN intrusion detection and redesigns the subsequent feature optimization and classification process accordingly. Specifically, the original features are fused with the outputs of multiple unsupervised detection modules to better characterize the intrinsic properties of the data. Based on the enhanced feature representations, a greedy feature vector (FV) selection strategy is employed to optimize the expanded feature space, reducing computational overhead while improving classification performance. Multiple FV selection schemes are used and evaluated to identify optimal feature subsets under limited computational resources. Finally, the optimized feature set is used to train a LightGBM. Ke et al. (2017) ensemble classifier for multiclass prediction. Based on Gradient Boosting Decision Trees (GBDT), LightGBM requires minimal data preprocessing while efficiently modeling complex feature interactions, making it particularly suitable for resource-constrained and heterogeneous distributed SDN environments. By integrating the original features with the enhanced representations generated by the unsupervised detection modules, the classifier acquires a more comprehensive and discriminative description of the data, thereby improving overall detection performance.

As illustrated in Fig. 2, the proposed framework consists of three stages. (1) Unsupervised representation learning: multiple unsupervised algorithms are applied to the original data to generate diverse feature representations. (2) Feature space fusion: informative features are selected from the generated representations and combined with the original features to achieve a balance between feature quality and computational cost. (3) Classification: the enriched feature space is used to train classifiers in a distributed manner, producing the final prediction model. The primary objective of the proposed framework is to adapt and optimize a supervised representation learning for resource-constrained, distributed Software-Defined Networking (SDN) environments. The framework follows a "local enhancement then global collaboration" workflow: unsupervised

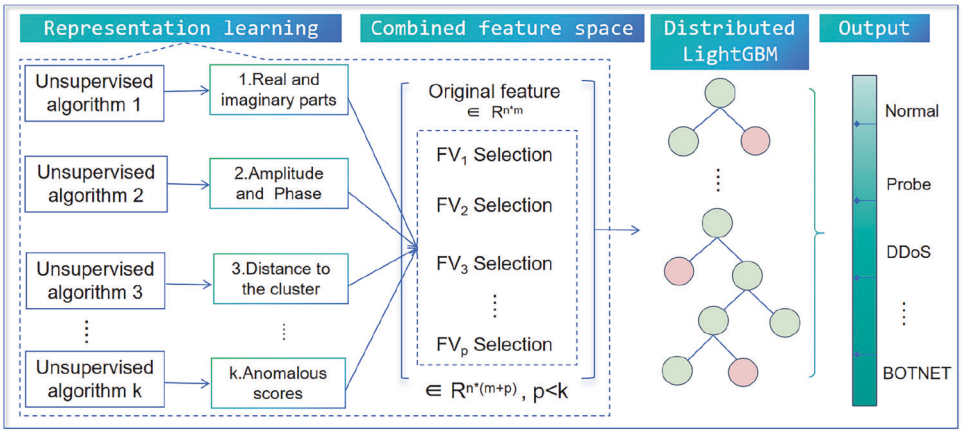


Fig. 2 Algorithm framework

representation learning enriches features locally, after which distributed LightGBM training is performed across nodes. This workflow aims to improve feature quality and then leverage parallel resources to reduce training time and communication overhead. Furthermore, several FV selection strategies are designed and experimentally validated to achieve an effective trade-off between feature quality and computational cost.

2.2 Representation learning phase

Representation learning, also known as feature learning, is a key subfield of machine learning. Its primary goal is to automatically learn efficient representations of raw data through algorithms, enabling extracting meaningful features for subsequent machine learning tasks Zhang et al. (2023). In this work, we focus on a supervised multi-class classification setting, where the learned representations are used to distinguish among. To formally define the setup for the multi-class classification problem, where the original feature space has n samples and m feature dimensions, and there are C class labels, $y \in \{1, 2, \ldots, c\}$ represents the class labels. Let L denote a set of labeled observations of X, which can be described as follows:

$$L = \{(x_i, y_i) | x_i \in \mathbb{R}^m, y_i \in \{1, 2, \ldots, C\}, i = 1, 2, \ldots, n_L\}$$

The original feature space X is mapped to feature vectors FV using an unsupervised algorithm ϕ. A good FV contains rich information from the original dataset X and enhances the generalization ability of the vector space. Based on the properties of ϕ, FV with different representational meanings and dimensions d can be generated.

$$\phi : X \rightarrow FV, \quad FV \in \mathbb{R}^{n \times d}$$

Here, ϕ can be any unsupervised algorithm. Thus, k different ϕ-mappings will generate k basic feature vectors $FV$from the original feature space X, each with a dimensionality of d. These k feature vectors form a new feature matrix space:

$$\phi = [\phi_1, \ldots, \phi_i, \ldots, \phi_k]$$

Ultimately, we obtain the following formula:

$$\begin{aligned} \phi(X) &= \left[\phi_1(X)^T, \ldots, \phi_i(X)^T, \ldots, \phi_k(X)^T\right] \\ &= [FV_1, \ldots, FV_k] \in \mathbb{R}^{n \times k} \end{aligned}$$

In this study, we explore using data generated and extracted by unsupervised anomaly detection algorithms as foundational feature transformation vectors. Notably, employing heterogeneous algorithms often yields superior results, as similar algorithms tend to produce consistent outputs with limited contributions to feature diversity. Our goal is to employ diverse algorithms to facilitate the learning of varied data features, thereby enriching information content and enhancing the generalization capability of the vector space. However, highly correlated algorithms may not only fail to improve predictions but also increase computational overhead, emphasizing the importance of algorithmic diversity. Additionally, the feature vector set extracted by unsupervised algorithms may contain components that enhance accuracy and potential sources of error.

Thus, improving the accuracy of feature vectors is critical. Inaccurate feature vectors can undermine predictive performance, making the balance between diversity and accuracy a key consideration. Against this backdrop, we adopt various types of unsupervised algorithms to extract feature vectors and fine-tune algorithm parameters to optimize accuracy further. This approach aims to ensure the reliability of feature vectors while maintaining diversity. This design not only improves the quality of feature representations but also provides a broader space for exploring enhancements in predictive performance.

2.3 Combined feature space phase

One significant advantage of using unsupervised representation learning for feature selection is its ability to increase feature diversity rapidly. In classification tasks, if some features fail to classify specific examples correctly, other features may successfully do so, improving the model's overall accuracy. Ensuring high feature diversity is therefore critical—an ensemble model where all features yield similar results offers limited practical value. To achieve this, it is essential to explore diverse approaches to feature generation. By employing various feature selection methods and validation perspectives, we can enhance the accuracy, diversity, and stability of the feature selection process.

After generating the feature matrix space $\phi(X)$ is merged with the original feature dataset X to create a Combined feature space (CFS). The resulting feature space consists of m original feature dimensions and $\sum_{k=1}^{n} d$ newly generated feature dimensions:

$$\mathrm{CFS} = [\mathrm{X}, \phi(X)] \in \mathbb{R}^{n \times \left(m + \sum_{i=1}^{k} \mathrm{d_i}\right)}$$

Not all $\phi_i(X)$ contribute equally to prediction tasks. Reducing the number of selected $\phi_i(X)$ in resource-constrained edge device scenarios accelerates execution by minimizing computational overhead on the original data. Therefore, it is necessary to identify the optimal CFS, which entails selecting the most effective X and $\phi(X)$. To achieve this, feature importance is utilized to select X, while three distinct feature selection methods are defined for $\phi(X)$. Let S represent the set of selected optimal $\phi(X)$, referred to as the optimal feature space. Below, we detail these three feature selection methods:

- **Random selection (RS):** In this method, we randomly select p features from $\phi(X)$ and add them to the set S, where $p < k$. Each randomly selected $\phi_i(X) \in \phi(X)$. The resulting optimal feature space is:

$$S = [\phi_1(X), \ldots, \phi_p(X)]$$

- **Accurate selection (AS):** We use feature importance to select X, and similarly, feature importance is used to select $\phi(X)$. The top p features with the highest importance are selected and added to the set S, where $p \leq k$. Feature importance is determined by the contribution of each $\phi_i(X)$ to the actual y values, following a greedy selection approach:

$$AS(i) = \text{feature_importance}\,(\varphi_i(X)^T, y), \mathrm{AS}(i) \geq 0$$

The resulting optimal feature space is:

$$S = [\phi_{a_1}(X), \ldots, \phi_{a_p}(X)], a_i \in \mathrm{TOP}_p(\mathrm{AS})$$

LightGBM provides two feature importance metrics: 'split' (usage frequency) and 'gain' (information gain). We adopt the default 'split' metric to rank feature contributions.

- **Balance selection (BS):** To balance diversity and accuracy, we select $\phi_i(X)$ by considering both feature importance and the correlation between features. To enhance the diversity of S, we define:

$$BS(i) = \frac{AS(i)}{\sum_{j=1}^{\Phi} \left|\rho(\varphi_i(X), \varphi_j(X))\right|}, BS(i) \geq 0$$

The resulting optimal feature space is:

$$S = [BS(1), \ldots, BS(p)], \quad p \leq k$$

Here, the Pearson correlation $p(\phi_i(X), \phi_j(X))$ is used to measure the correlation between a pair of features (ϕ_i, ϕ_j). The correlation between $\phi_i(X)$ and all previously selected $\phi_j(X)$ in S is aggregated as:

$$\sum_{\phi_j(X) \in S_{t-1}} \left|\rho(\phi_i(X), \phi_j(X))\right|$$

If $\phi_i(X)$ is highly correlated with any $\phi_j(X)$ already in S, the denominator in the equation becomes larger. High correlation indicates overlapping information, so this approach reduces the likelihood of selecting features with significant redundancy. Each time, the top p features with the highest $BS(i)$ values are selected, and the feature with the maximum $BS(i)$ is added to S. This process continues until the size of S equals p. The workflow is detailed in Algorithm 1.

Algorithm 1 Balance Select Algorithm

Require: $\phi(X) \in \mathbb{R}^{n \times m}$ (Feature matrix); $importance \in \mathbb{R}^{m}$ (Feature importance); p (Number of features to select)
Ensure: $S \in \mathbb{R}^{n \times p}$ (Selected features)
Initialize $indices \leftarrow []$ ▷ Stores indices of selected features
$n_features \leftarrow$ number of columns in $\phi(X)$
Step 1: Select the first feature with the highest importance
$max_idx \leftarrow \text{argmax}(importance)$
Append max_idx to $indices$
$importance[max_idx] \leftarrow -1$ ▷ Mark as selected
Step 2: Iteratively select remaining $p - 1$ features
for $iter = 1$ to $p - 1$ **do**
  $scores \leftarrow \mathbf{0} \in \mathbb{R}^{n_features}$
  for each unselected feature j **do**
    $total_corr \leftarrow 0$
    for each selected index s in $indices$ **do**
      $total_corr \leftarrow total_corr + |\text{pearsonr}(\phi[:, j], \phi[:, s])|$
    end for
    if $total_corr = 0$ **then**
      $scores[j] \leftarrow \infty$ ▷ No redundancy, prioritize selection
    else
      $scores[j] \leftarrow \frac{importance[j]}{total_corr}$
    end if
  end for
  $max_idx \leftarrow \text{argmax}(scores)$
  Append max_idx to $indices$
  $importance[max_idx] \leftarrow -1$ ▷ Mark as selected
end for
Step 3: Extract the selected features
$S \leftarrow \phi[:, indices]$
return S

To clarify the computational overhead of the balanced selection strategy, we compare its complexity with that of the accurate selection method. The accurate selection requires only a single sorting of precomputed importance scores, resulting in $O(m \log m)$ time, where m is the number of candidate features. In contrast, the balanced selection performs incremental recomputation: at iteration k (with k features already selected), each remaining candidate's score is recomputed by summing its absolute Pearson correlations with all k selected features, costing $O(m \cdot k \cdot n)$ for that iteration, where n is the sample size. Summing over $p - 1$ iterations yields an overall complexity of $O(mnp^2)$. In our experiments, since p is typically small (e.g., $p \leq 20$), this additional cost remains manageable, yet the analysis highlights the trade-off between predictive benefit and computational efficiency.

Ideally, a feature selection system should evaluate the quality of selected features independently of any classifier. However, because different methods and metrics yield varying feature subsets, each subset may represent a different local optimum in the feature space. This diversity underscores the importance of employing multiple feature selection algorithms. Such a diversified strategy not only enhances the model's robustness but also ensures the comprehensiveness and effectiveness of the feature selection process, leading to a more holistic evaluation.

2.4 Distributed collaborative classification stage

Tree-based models are widely recognized for offering an excellent balance between predictive performance, deployment cost, and operational stability. Among them, LightGBM is a gradient boosting decision tree (GBDT) framework designed for high training efficiency, low memory consumption, and scalable learning on large-scale datasets. These characteristics make it particularly well-suited for resource-constrained environments such as embedded and edge devices. Furthermore, LightGBM provides native support for distributed training. During the construction of each decision tree, model information is synchronized across distributed nodes to ensure the generation of a single globally consistent model. Specifically,

each node computes local gradients and Hessians from its local data and constructs corresponding local histograms. These histograms are then aggregated across nodes to determine the globally optimal split point. Once the split is identified, the tree structure and model parameters (e.g., leaf values) are synchronously updated across all participating nodes. The communication complexity of this process is $O(M \times m \times B/num)$, where M denotes the number of trees, m is the number of features, B is the number of histogram bins, and num represents the number of distributed nodes. Since the communication cost depends only on the feature dimension and histogram size rather than the number of training samples, LightGBM effectively eliminates communication bottlenecks for large-scale datasets while achieving near-linear scalability. Moreover, the mathematical equivalence between distributed and single-machine training guarantees identical model structures under the same training configuration. LightGBM also supports three distributed learning strategies—data parallelism, feature parallelism, and voting parallelism. Collectively, these practical advantages make LightGBM a highly reliable classifier for engineering applications and constitute the primary reason for selecting it as the final classifier in this study.

The fused feature space, denoted as $CFS = [X, S] \in \mathbb{R}^{n\times(d+p)}$, is subsequently fed into the LightGBM classifier to generate prediction results. To address class imbalance, LightGBM supports sample-level and class-level weighting through the sample_weight and class_weight parameters, thereby improving the robustness of the trained model. As a decision tree-based algorithm, LightGBM is inherently insensitive to feature scaling, eliminating the need for feature standardization or normalization while preserving the interpretability of the original feature representation. This property is particularly advantageous in Software-Defined Networking (SDN) applications, where the learned decision trees can be directly translated into network control logic and mapped to executable SDN flow table rules, facilitating practical deployment in real-world systems. In addition, LightGBM incorporates a built-in feature importance evaluation mechanism, enabling effective feature pruning during training to further improve detection accuracy and computational efficiency.

To further enhance classification performance, multiple unsupervised learning algorithms are employed to enrich the feature space and increase feature diversity. The resulting feature representations are subsequently integrated to improve the robustness of the classifier. This ensemble feature construction strategy can be regarded as an effective variance-reduction approach, as the combination of heterogeneous feature mappings $\phi_i(X)$ captures complementary structural information that is difficult for any single algorithm to model. Nevertheless, not all generated feature mappings are equally informative. Some $\phi_i(X)$ may contain noisy or inaccurate representations, introducing unreliable information into the fused feature space and consequently increasing the overall bias of the model. This phenomenon explains why directly incorporating all generated features may degrade predictive performance. To address this issue, a feature selection mechanism is applied to retain only the most informative feature mappings $\phi_i(X)$, thereby reducing redundant and biased information, improving the quality of the fused feature space, and ultimately enhancing the overall classification performance.

2.5 Distributed multi-domain networking architecture

Software-Defined Networking (SDN) has emerged as a flexible network architecture and is being increasingly adopted in cloud computing infrastructures, software-defined wide area networks (SD-WANs), Internet of Things (IoT) systems, and edge networks. As network scale increases, a single control domain may no longer provide sufficient scalability or management efficiency. Large-scale deployments, therefore, commonly adopt a multi-domain SDN architecture, in which each domain is independently managed by its own control plane. Based on the organizational structure of these control planes, multi-domain networking architectures can be broadly classified into horizontal and vertical models, as illustrated in Fig. 2. Accordingly, a distributed framework intended for large-scale SDN environments should support both architectural models.

Horizontal networking architecture (Fig. 3): A horizontal SDN architecture deploys multiple peer control planes to enable scalable network management and efficient resource utilization. Each controller independently manages a designated geographical area or functional module. Horizontal multi-controller systems are generally implemented using either distributed controller clusters or east-west interface protocols. A distributed cluster enables homogeneous controller instances to coordinate and share network state, but it typically provides limited support for collaboration among heterogeneous controllers. In contrast, east-west interfaces facilitate interoperability across heterogeneous control domains by exchanging routing, topology, and network-state information, thereby supporting inter-domain communication. This architecture is particularly suitable for micro-data centers in edge computing environments, local communication among embedded devices, and coordination among base stations in mobile networks. Its main advantages are threefold. First, geographically distributed controllers reduce

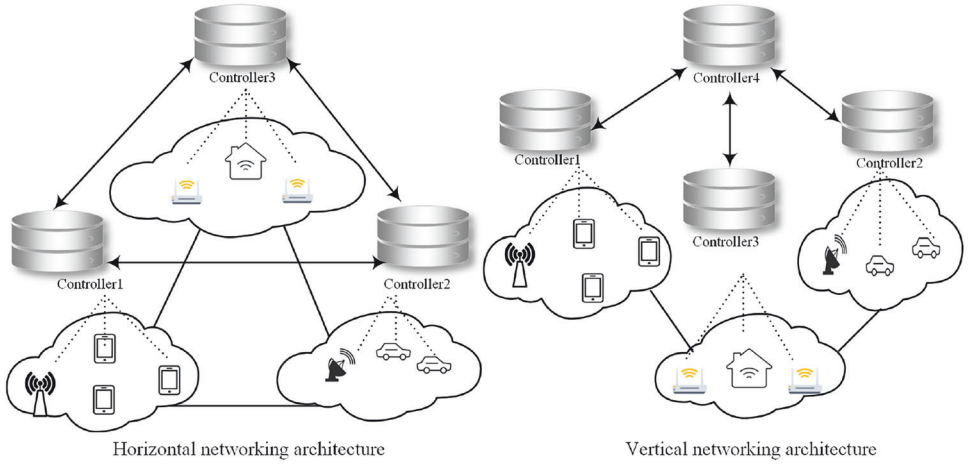


Fig. 3 Distributed Multi-Domain Networking Architecture

southbound communication latency, enabling embedded devices and edge nodes to respond rapidly to control instructions. Second, east-west interfaces improve interoperability among controllers operating on different hardware platforms or protocol standards. Third, multiple peer controllers provide redundancy and fault tolerance, allowing essential network functions to remain available even when an individual controller fails. Given its widespread application in embedded, mobile, and edge networks, the horizontal SDN architecture was chosen for validation in the experiments detailed in Section 2.2.

Vertical networking architecture (Fig. 3): In contrast to the peer-based horizontal model, a vertical SDN architecture adopts hierarchical control to combine unified global management with rapid local response. It typically consists of a super-controller layer and a domain-controller layer. The super-controller is responsible for cross-domain communication, global policy enforcement, and network-wide resource coordination, whereas each domain controller manages local devices, traffic scheduling, and forwarding decisions within its assigned domain. Each domain controller presents its managed infrastructure as an abstract and independent network domain to the super-controller, allowing the latter to maintain a unified global view. For example, in an embedded-device network for smart manufacturing, the super-controller may coordinate communication and resource allocation across production areas, while domain controllers locally manage latency-sensitive device connectivity and optimize data flows. This division of responsibilities improves both local responsiveness and overall system efficiency. The vertical architecture, therefore, provides a scalable hierarchical control solution for complex embedded, mobile, and edge networking environments, offering autonomous domain management, efficient division of control functions, and coordinated network-wide operation.

2.6 Evaluation matrix of model results

This study uses four metrics to evaluate detection performance: accuracy, precision, recall, and F1-score.

Accuracy represents the proportion of correctly predicted instances, calculated as $\frac{TP+TN}{TP+TN+FP+FN}$. It measures the overall correctness of the model's predictions.

Precision indicates the proportion of actual attacks among the traffic classified as attacks, expressed as $\frac{TP}{TP+FP}$. It evaluates the "purity" of anomaly detection, minimizing the tendency to classify legitimate traffic as malicious falsely.

Recall measures the proportion of actual attacks correctly identified as attacks, computed as $\frac{TP}{TP+FN}$. It reflects the model's ability to comprehensively detect anomalies, reducing missed detections.

F1-Score, the harmonic mean of Precision and Recall, is given by $\frac{2\times TP}{2\times TP+FP+FN}$. It balances precision and recall, serving as a comprehensive measure of model performance.

These metrics are derived using four key components: True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). TP: The number of network traffic instances that are actual attacks and correctly identified by the model. This reflects the successful detection of anomalous traffic. TN: The number of normal traffic instances accurately classified as normal by the model, indicating successful recognition of legitimate traffic. FP: The number of normal traffic instances incorrectly classified as attacks, representing legitimate traffic mistakenly flagged as malicious. FN: The number of actual attacks wrongly classified as normal, indicating missed detections of anomalous traffic that should have been identified.

3 Experimental design

This study investigates the potential of lightweight ensemble models for intrusion detection in SDN environments

under a distributed framework, focusing on how unsupervised representation learning can enhance supervised learning models. For experiment, used an Intel® i7-7500U processor with a clock speed of 2.70 GHz and 8 GB of memory. No GPU was utilized, in order to better align with the hardware conditions of typical daily network environments. To simulate a realistic distributed SDN control plane environment, three virtual machines (VMs) were created using VMware virtualization software. Each VM was installed with Ubuntu 20.04 LTS and allocated 2 GB of memory. These three VMs simulate three SDN controller nodes, with Ryu acting as the controller, and the unsupervised augmentation and distributed collaborative framework from Section 2.1 running as the application for the controller. The corresponding experimental design is presented in this section.

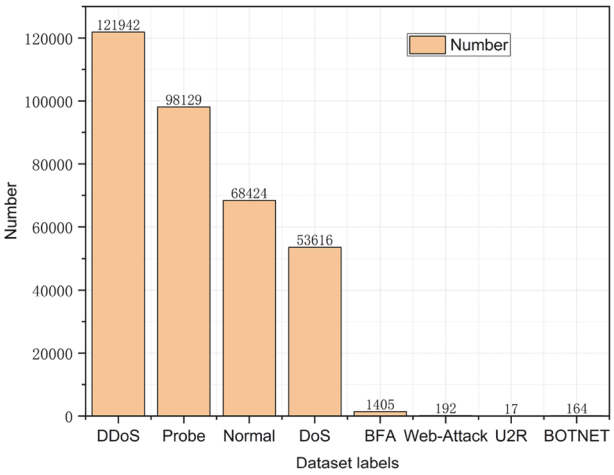


Fig. 4 Overview of the Public Dataset InSDN

3.1 Feature space description

To comprehensively evaluate the generalization capability of the learned representations across diverse application scenarios, seven publicly available benchmark datasets with varying feature dimensionalities, sample sizes, and class distributions were selected for cross-domain validation. Specifically, the Arrhythmia dataset (452 samples, 274 features) originates from electrocardiogram-based cardiac diagnosis; the Cardio dataset (1600 samples, 32 features) is designed for cardiovascular disease risk assessment based on cardiac imaging parameters; the Letter dataset (1831 samples, 21 features) represents a typical multiclass optical character recognition task; the Mammography (MG) dataset (3686 samples, 600 features), derived from breast X-ray imaging, exhibits a highly imbalanced class distribution; the MNIST dataset (6435 samples, 36 features) contains handwritten digit images; the Satellite dataset (7603 samples, 100 features) is generated from Landsat multispectral remote sensing imagery; and the Speech dataset (11,863 samples, 6 features) is constructed from speech signal characteristics. Collectively, these datasets span multiple domains, including medical diagnosis, image recognition, remote sensing analysis, and speech processing, thereby providing a comprehensive benchmark for assessing the robustness and transferability of the proposed approach under heterogeneous data conditions.

Furthermore, to enhance the reproducibility of the experimental results and facilitate fair comparisons with existing studies, the publicly available InSDN dataset (343,889 samples, 83 features, and 8 classes) was adopted in place of the self-collected dataset (Elsayed et al. 2020). This choice mitigates the reproducibility limitations commonly associated with proprietary datasets. As illustrated in Fig. 4, the InSDN dataset exhibits a pronounced class imbalance. During the preprocessing stage, a stratified random sampling strategy was employed to partition the complete dataset (D) into three mutually exclusive subsets ($D_{m}{}_{m=1}^{3}$), while preserving the original class distribution within each subset. To ensure sufficient data availability for local model training, the size of each subset was constrained to be no less than 20% of the original dataset. Subsequently, the three subsets were deployed on three independent SDN controller nodes, respectively, thereby constructing a distributed control-plane environment for experimental evaluation.

The performance of the LightGBM model largely depends on the representational quality of the input features. To enhance the diversity and information richness of feature representations, multiple unsupervised learning methods were employed to augment the original feature space. Specifically, we did not actively select among the seven unsupervised feature extraction algorithms; this decision was mainly based on the consideration of improving detection performance and enhancing generalization capability. Accordingly, no dedicated hyperparameter tuning or method-specific selection was performed. Instead, the emphasis was placed on enriching the diversity and representational capacity of the resulting feature space. Randomly selecting representative algorithms from different methodological categories can improve model robustness by reducing dependence on any particular feature extraction strategy. Therefore, one representative algorithm was methodological category coverage from each of seven categories of unsupervised methods, namely Local Outlier Factor (LOF), Nearest Neighbors (k-NN), Isolation Forest, Principal Component Analysis (PCA), Uniform Manifold Approximation and Projection (UMAP), K-Means clustering, and spectral methods. For brevity, the implementation details of these algorithms are omitted. Collectively, these algorithms encompass a broad spectrum of feature modeling mechanisms, including

density- and distance-based approaches, spectral methods, matrix factorization and linear techniques, statistical and probabilistic methods, as well as non-parametric anomaly-scoring approaches. Together, they generated 141 newly constructed features for each dataset, substantially enriching the original feature representation. Since each method possesses distinct strengths and limitations, combining representative algorithms from different methodological categories introduces diversity into the feature generation process while maintaining systematic feature construction, thereby improving the overall predictive performance of the model. It should be emphasized that the seven algorithms were not selected to maximize performance on any specific dataset. Instead, one representative algorithm was chosen from each methodological category (density-based, distance-based, spectral, matrix factorization, etc.) to improve representational diversity and reduce selection bias. This design choice aims to enhance generalization across different data distributions at the cost of potential suboptimality on any single dataset —a trade-off we consider acceptable given the cross-domain validation objective of this study.

3.2 Experimental setup for distributed collaborative training

The objective of the collaborative experiment is to develop a distributed training framework that satisfies the following requirements: (1) efficiently schedules computational resources through distributed execution; (2) enables each worker node to maintain a fully functional local model; and (3) improves classification performance as network traffic is processed collaboratively by an increasing number of sub-models. By leveraging multiple computing nodes, the proposed framework effectively utilizes distributed resources while eliminating the computational bottlenecks associated with a single node.

To ensure compatibility with the cluster environment, LightGBM was compiled from source using CMake, and the generated binaries were optimized for the target platform. The manually compiled executables can be deployed directly on each node without requiring a Python runtime, relying only on the C++ runtime environment. This lightweight deployment is well suited for high-performance distributed systems with strict resource constraints. Based on this implementation, distributed training was performed across all nodes in the cluster. Each node trained on its local dataset and independently computed gradients and histograms, allowing all nodes to collaboratively determine the globally optimal split points without exchanging raw data.

During training, the master node and worker nodes collaboratively construct the global model. Each worker node first computes local gradients and histograms independently before transmitting the results to the master node. The master node aggregates the histograms from all workers to identify the globally optimal split point and then broadcasts the selected split information back to the worker nodes. Based on the received split point, each worker updates its local model accordingly. To maintain model consistency, the master node synchronizes the model states of all worker nodes after each iteration, thereby reducing the bias introduced by data partitioning. This collaborative mechanism enables efficient inter-node communication without sharing raw data, allowing all nodes to jointly train a unified global intrusion detection model. In this process, the master node receives aggregated gradients and histograms from the worker nodes, which are statistical summaries rather than raw data.

Furthermore, LightGBM supports training resumption through the init_model parameter, enabling interrupted training tasks to continue from the checkpoint stored by the master node. Owing to the globally synchronized training process, the saved model can be directly used for inference or further training without requiring any additional model-merging procedures. This capability enhances the flexibility and reliability of the proposed framework in practical engineering applications.

The distributed experiments support both horizontal and vertical network architectures. In both configurations, computational tasks are distributed across multiple nodes for parallel execution, thereby maximizing resource utilization. For experimental validation, three Ubuntu-based computers were configured as distributed nodes (controllers) and interconnected to form a horizontal network architecture. The newly generated feature-space representations, denoted as $\phi_i(X)$, were also learned independently on their respective nodes.

The experiments were built upon LightGBM's native distributed training mechanism. Prior to training, the datasets and configuration files were prepared on each node. After configuration, distributed training was initiated using the corresponding execution commands. The GOSS sampling strategy and the data-parallel tree learner were adopted as the standard LightGBM configurations to improve training efficiency. Considering that the dataset contains substantially more samples than features, the data-parallel strategy was selected. Detailed implementation procedures can be found in the official LightGBM documentation[1]. The hyperparameters summarized in Table 1 were determined through grid search. By fully exploiting LightGBM's native distributed computing capabilities for resource scheduling, the proposed

[1] https://github.com/microsoft/LightGBM/blob/master/examples/parallel_learning/README.md

Table 1 Hyperparameter Configuration

Parameter Name	Parameter Value	Parameter Name	Parameter Value
random_state	42	min_child_weight	0.3
objective	multiclass	num_leaves	150
num_class	9	data	Triandata1.csv
class_weight	balanced	output_model	model.txt
boosting_type	gbdt	tree_learner	data
data_sample_strategy	goss	local_listen_port	12400
subsample	0.9	num_machines	3
colsample_bytree	0.8	controller1	192.168.1.101
learning_rate	0.0305	controller2	192.168.1.102
max_depth	15	controller3	192.168.1.103
min_child_samples	15		

Table 2 Unsupervised representation learning performance (F1 scores) across datasets

Dataset	Original	New	Fused	Bal-k	Balanced	Acc-k	Accurate	Rnd-k	Random
arrhythmia	0.7760	0.7451	**0.7766**	30	0.7405	20	0.7417	130	0.7617
cardio	0.9519	0.9452	0.9549	40	0.9533	25	0.9543	40	**0.9570**
letter	0.7874	0.7816	0.8207	35	0.8144	45	0.8246	55	**0.8305**
MG	0.7704	0.7635	0.7826	20	0.7712	25	**0.7895**	20	0.7723
mnist	0.9428	0.8600	**0.9448**	120	0.9445	80	0.9445	105	0.9401
satellite	0.8892	0.8931	0.8951	35	0.8970	50	**0.8975**	45	0.8970
speech	0.5236	0.5500	0.5297	25	**0.5623**	20	0.5334	50	0.5219
InSDN	0.9686	0.9531	0.9854	28	0.9945	37	0.9939	28	**0.9962**

The bold values in table indicate the best performance result for each dataset among all compared methods

framework efficiently completed the training of the classifier.

This paper aims to verify the feasibility and performance of the proposed method. Due to resource constraints, we did not conduct further system performance tests under varying node scales, data volumes, or heterogeneous hardware environments. Nevertheless, optimizing and adapting distributed resource scheduling performance remains an important direction for future research in SDN.

4 Experimental results and analysis

To comprehensively evaluate the enhancement effect of unsupervised representation learning on supervised learning, we conducted the following experiments: analysis of varying feature quantities in the original feature space X, the new feature space ϕ, and the Combined feature space (CFS) with different subsets of the new feature space (unsupervised representation-enhanced subsets). Additionally, to further reduce feature dimensions for improved data transmission and computational efficiency while maintaining classification performance, we evaluated the optimal combination of subsets from the original and new feature spaces (optimal Combined feature space). Details are discussed below.

4.1 Overall Performance Analysis of Unsupervised Representation Learning

In intrusion detection, which is a form of anomaly detection, Table 2 summarizes the F1-score performance of the proposed representation learning method across seven commonly used anomaly detection datasets[2] and the InSDN dataset. The datasets are divided into training (75%) and testing (25%) sets. The results include the model trained on the original data, the model trained on generated data, and the model trained on the fused dataset. Additionally, three sample selection strategies—Balanced selection, Accurate selection, and Random selection—along with their corresponding optimal (k) values are evaluated.

Overall, the fused representation consistently outperforms the models trained only on original or generated data, indicating that real and generated samples are complementary. For example, on the letter dataset, the fused F1-score improved from 0.7874 of original and 0.7816 of

[2] Available: http://odds.cs.stonybrook.edu(ODDS Library)

generated to 0.8207. Similar improvements were observed on the mammography from 0.7704 to 0.7826 and satellite from 0.8892 to 0.8951 datasets. These results suggest that combining generated features with original data effectively enhances the model's generalization capability. In the field of intrusion detection, this method can improve the model's ability to generalize. Particularly in high-traffic network environments, even a small statistically significant performance improvement can lead to substantial and far-reaching security enhancements after large-scale deployment and long-term operation, resulting in cumulative defense effectiveness.

For the three sample selection strategies, the optimal (k) value varies across datasets. The Balanced strategy performs well on several datasets, for instance, achieving an F1-score of 0.8970 with a (k) value of 35 on the satellite dataset. The Accurate strategy also performs well and even outperforms the others on certain datasets. For example, on the mammography dataset, the Accurate strategy achieves the highest F1-score of 0.7895 at $k = 25$. Meanwhile, the Random strategy also performs well on certain datasets, such as the letter dataset, where it achieved an F1-score of 0.8305. This suggests that the model is relatively robust to random variations in the samples when sufficient informative samples are included.

Notably, the proposed representation learning shows improvement on the InSDN dataset, where the fused dataset achieved an F1-score of 0.9854. The Random and Balanced strategies increase the F1-score to 0.9962 and 0.9945, respectively. These results demonstrate the effectiveness of the method in large-scale network traffic anomaly detection tasks.

In conclusion, the experimental results indicate that combining generated samples with the original dataset generally improves detection performance across multiple domains. Additionally, appropriate sample selection strategies can further optimize the model, although the best configuration depends on the characteristics of each dataset. To strike a balance, the following section presents a more detailed analysis of the InSDN dataset.

4.2 Analysis of the original feature space

We first investigated the effectiveness of feature selection in the original feature space X. The Accurate Selection (AS) method introduced in Section 2.3 was employed to rank the original features according to their importance scores, and features were incrementally added in descending order. Although the InSDN dataset contains 82 features, several features with low importance scores were excluded to reduce redundancy and computational complexity. As shown in Fig. 5, the model performance strongly depends on the selected feature subset. When only a small number of highly ranked features were retained, the limited feature representation was insufficient to characterize the complex traffic patterns, resulting in relatively low detection performance. By gradually increasing the number of selected features, the model obtained more discriminative information and achieved stable performance when approximately 28 features were included. At this point, the Accuracy, Precision, F1-score, and Recall reached 0.9961, 0.9842, 0.9791, and 0.9750, respectively, indicating that further increasing the number of selected features provided only limited additional performance improvement. Beyond this point, additional original features provided limited improvements, indicating that most useful information had already been captured by the selected subset.

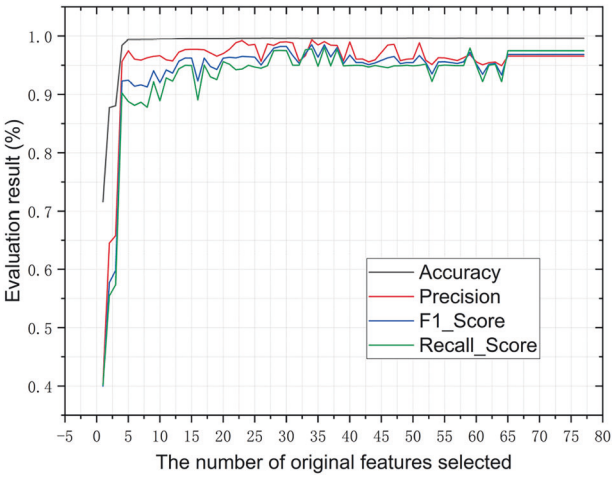


Fig. 5 The number of original features selected: Model performance (Accuracy, Precision, F1-score, and Recall) on the InSDN dataset as a function of the number of selected original features. Features are ranked by importance using Accurate Selection (AS) and added incrementally in descending order

These results establish the performance baseline of the original feature space and suggest that appropriate feature selection can effectively remove redundant information while maintaining detection capability.

4.3 Analysis of the new feature space

To evaluate whether unsupervised representation learning can extract additional discriminative information, we further analyzed the generated feature space $\phi(X)$ containing 141 dimensions. The same feature selection strategy was applied to investigate whether the learned representations could independently support classification. As illustrated in Fig. 6, the generated feature space achieved competitive detection performance after selecting an appropriate subset of representations. For example, when 28 new features were selected, the model achieved an Accuracy, Precision, F1-score, and Recall of 0.8561, 0.7372, 0.6711, and 0.7014,

respectively. Compared with the original feature space, the selected $\phi_i(X)$ features demonstrate that unsupervised representation learning can transform the original traffic characteristics into a more compact and informative representation space.

However, the performance improvement is not strictly proportional to the number of generated features. This phenomenon indicates that although unsupervised learning can capture latent patterns from network traffic, some generated representations may contain overlapping or less discriminative information. Therefore, selecting informative representations remains necessary to avoid unnecessary computational costs.

A comparison between Figs. 5 and 6 shows that the learned representations can partially replace manually designed features while preserving classification capability. Nevertheless, the complementary relationship between original features and generated representations motivates the following analysis of their combined feature space.

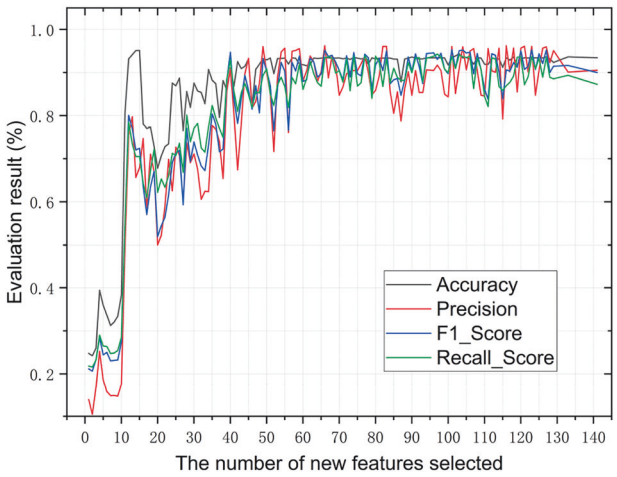


Fig. 6 The number of new features selected: Model performance on the InSDN dataset as a function of the number of selected features from the unsupervised representation space $\phi(X)$. Features are selected using Accurate Selection (AS) from the 141 generated dimensions

4.4 Analysis of unsupervised representation enhancement under different feature selection strategies

This section evaluates the enhancement of supervised learning by unsupervised representation learning, specifically integrating different subsets of new feature space into the Combined feature space (CFS). As described in Section 2.3, the selection of $\phi_i(X)$ subsets increases based on three strategies: balanced selection, accurate selection, and random selection. The results for different strategies and varying numbers of $\phi_i(X)$ are analyzed below. Notably, when $p < 0$, the default settings of the LightGBM model are used. For $p \geq 0$, the model employs tuned parameters specified in Table 4. Here, $p = 0$ refers to the scenario where only the original features are used without additional feature subsets. Each case is discussed below, with all graphs scaled uniformly to a range of 6 for clarity.

As shown in Fig. 7 (accurate_select), the performance for $p > 1$ generally surpasses that for $p = 0$, demonstrating that precise selection can effectively enhance model performance by identifying informative features. Although the tuned LightGBM already captures the information contained in the original features effectively, incorporating selected unsupervised representations provides further improvement. The performance gain is particularly evident when only a small number of additional features are introduced, whereas the marginal benefit gradually diminishes as p increases. This suggests that additional features contribute progressively less discriminative information and may eventually introduce redundancy. Therefore, in practical applications, restricting the number of added features to approximately 1–6 provides a favorable trade-off between predictive performance and computational complexity.

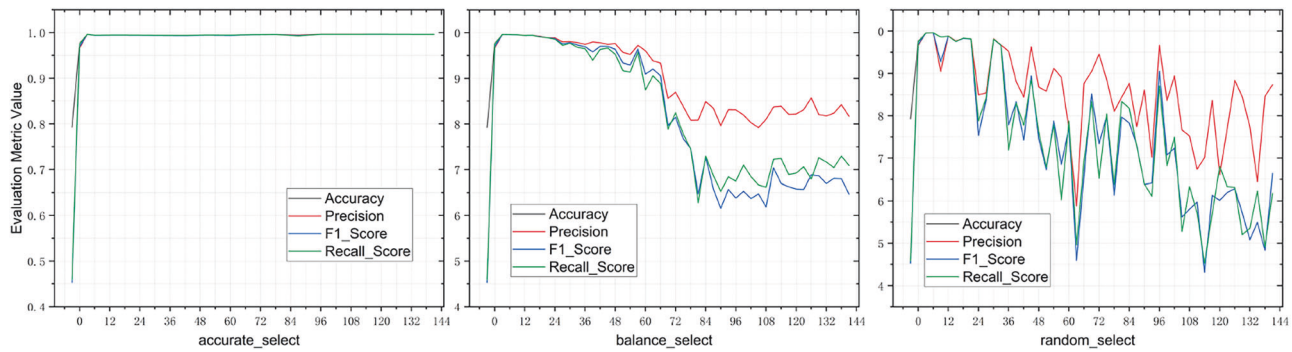


Fig. 7 Comparison of Model Performance Metrics under Different Feature Selection Strategies: Performance comparison of Accurate Selection, Balanced Selection, and Random Selection on the InSDN dataset. The horizontal axis p denotes the number of added $\phi_i(X)$ features. For $p < 0$, default LightGBM parameters are used; for $p \geq 0$, the tuned parameters in Table 4 are applied. The dashed line ($p = 0$) indicates the baseline using original features only

As shown in Fig. 7 (balance_select), the first 1-18 selected features generally improve performance, but adding beyond this range leads to deterioration. This indicates that balanced selection is effective when the number of added features is controlled, whereas excessive redundancy or noise can interfere with learning. However, balanced selection is not universally superior to accurate selection; its effectiveness hinges on the structure of the candidate feature space. It excels when the space is large and contains substantial redundancy or highly correlated representations, where individual-performance-based selection tends to produce homogeneous subsets—whereas balanced diversity preserves complementary information. Conversely, in a well-discriminated space where a few top features already provide sufficient complementarity, imposing redundancy constraints offers limited benefit and may exclude strong predictors. Given that the tuned LightGBM already extracts substantial information from the original features, the marginal gain of any expansion is inherently bounded. Therefore, strategy selection should align with data distribution, and excessive or ill-suited inclusion risks weakening informative contributions and degrading performance.

Figure 7 (random_select) exhibits a different pattern. Although selecting $p > 1$ can yield improvements in some cases, particularly when $p < 6$, the performance fluctuates considerably as the number of selected features changes. This variability reflects the inherent randomness and uncertainty introduced by random feature selection and indicates that increasing the feature count does not guarantee consistent performance gains. Nevertheless, the effectiveness of a feature subset is also dataset-dependent. Random selection may occasionally identify features that happen to align well with the underlying data distribution and can therefore produce competitive or unexpectedly favorable results despite the absence of an explicit optimization criterion. For the InSDN dataset, however, precise and balanced selection provide more reliable strategies for optimizing the expanded feature space, whereas random selection remains less predictable and requires further investigation to establish its robustness and general applicability.

Overall, Fig. 7 illustrates the performance of LightGBM when the original feature space is expanded with varying numbers of $\phi_i(X)$. The results demonstrate that even a small number of informative unsupervised representations can complement the original features and further improve an already well-tuned classifier. Consequently, useful $\phi_i(X)$ can contribute to prediction even when they constitute only a small proportion of the expanded feature space.

Notably, the three selection strategies exhibit distinct performance patterns: accurate and balanced selections yield generally stable improvements, whereas random selection produces considerable fluctuations and less predictable outcomes. This contrast implies that the choice of feature selection strategy should be based on the class distribution of the target dataset and the evaluation metrics of concern in practical applications, rather than presupposing that one strategy is optimal in all scenarios.

Beyond the strategy itself, these findings also demonstrate that not all unsupervised methods necessarily produce useful representations within a limited feature space. Nevertheless, appropriately selected unsupervised representations can enhance supervised classification on high-dimensional datasets. The key factor is therefore not simply expanding the feature space, but identifying representations that provide complementary discriminative information. Including excessive redundant or noisy features offers little additional benefit and may even aggravate the adverse effects of data imbalance or interfere with model learning; such features should therefore be excluded from the final feature subset.

Notably, unsupervised representation learning can still provide additional gains even when the baseline model has already achieved strong performance after parameter tuning. These improvements, however, come at a non-negligible computational cost, which may in some cases exceed that required for feature selection itself. Practical deployment therefore requires an appropriate balance between predictive improvement and computational efficiency to maintain the feasibility and scalability of the proposed framework. In this context, distributed computing provides an effective means of coordinating computational resources and alleviating the additional computational burden introduced by representation learning.

4.5 Optimal fusion matrix space analysis

To maintain high classification performance while reducing the time cost of generating additional spatial features, we further examined how different combinations of original features and newly generated features affect the final performance. Specifically, we selected five subset sizes from the original feature space, namely 5, 10, 15, 20, and 25, and six subset sizes from the new feature space, namely 3, 6, 9, 12, 15, and 18. This produced a total of 30 feature combinations.

For each combination, we constructed a fused feature set by combining a fixed number of original features with a variable number of new features, and then evaluated the classification performance using three feature selection strategies: Random Selection (RS), Accurate Selection (AS), and Balance Selection (BS). In RS, (p) features are randomly selected from the candidate feature set. In AS, the top-(p) features are selected according to LightGBM feature importance. In BS, both feature importance and feature redundancy are considered, so that features with high

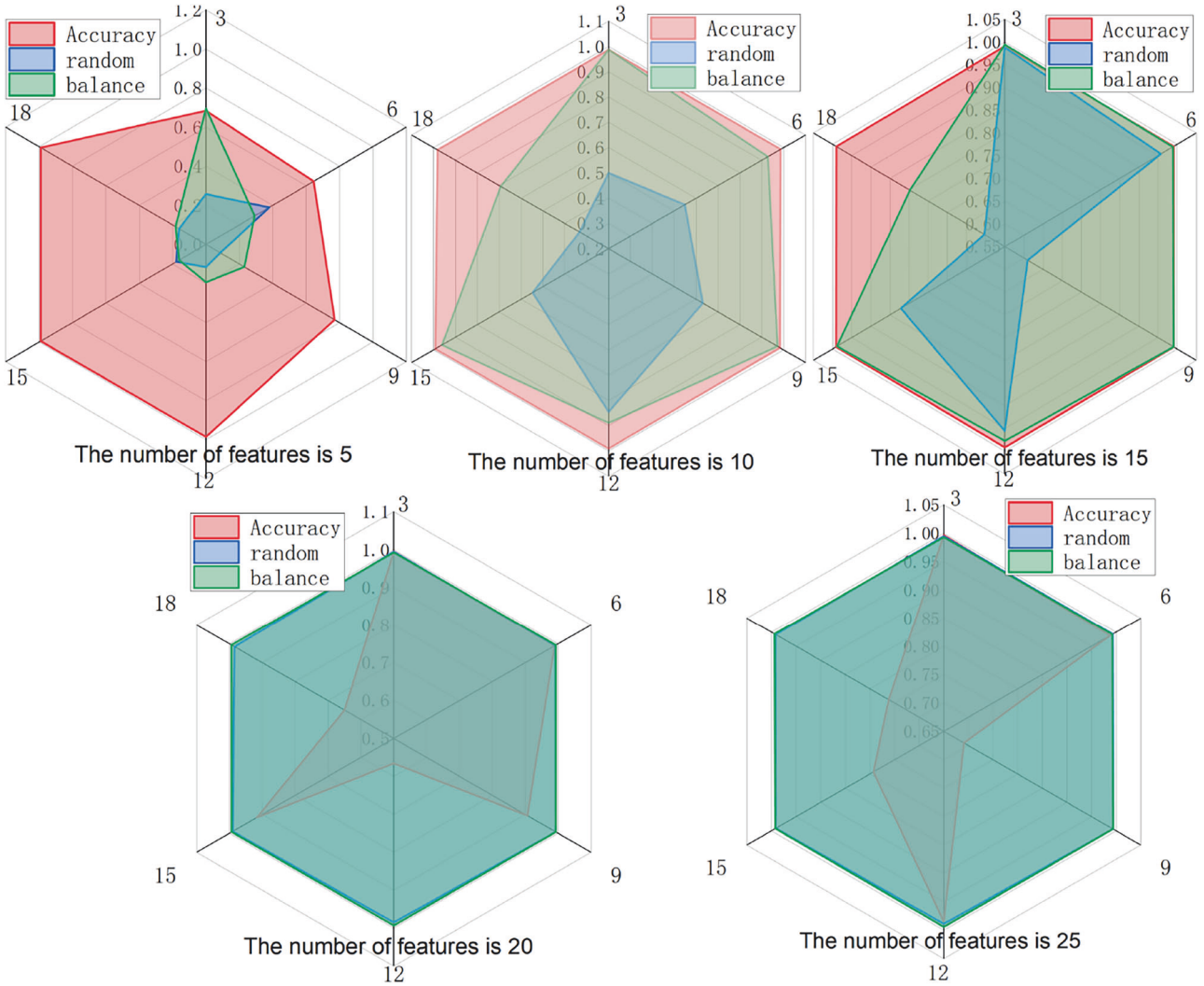


Fig. 8 Optimal combined feature space analysis: F1-score of different original–new feature combinations under three selection strategies on the InSDN dataset. Each radar chart fixes the number of original features (5, 10, 15, 20, or 25). The six axes represent the number of added new features (3, 6, 9, 12, 15, 18). Red, blue, and green areas correspond to Accurate, Random, and Balanced selection, respectively

importance but low correlation with already selected features are preferred.

Figure 8 shows the F1-scores of these combinations. Each radar chart fixes the number of selected original features and varies the number of selected new features. For example, in the upper-left chart, the number of original features is fixed at 5, while the six axes correspond to adding 3, 6, 9, 12, 15, and 18 new features, respectively. Therefore, this chart represents the six combinations $(5+3)$, $(5+6)$, $(5+9)$, $(5+12)$, $(5+15)$, and $(5+18)$. The red, blue, and green areas represent AS, RS, and BS, respectively, and the radial value indicates the corresponding F1-score.

The same procedure is applied to the other four radar charts, where the numbers of original features are fixed at 10, 15, 20, and 25, respectively. In this way, Fig. 8 provides a direct comparison of how the number of original features, the number of new features, and the feature selection strategy jointly influence classification performance. Since F1-score provides a balanced and comprehensive evaluation of the classifier, only this metric is reported here to avoid redundant presentation.

As illustrated in the five subplots of Fig. 8, the F1-Score demonstrates increased stability as the number of original and new features grows. The Accuracy method (red region) achieves a broad performance coverage and high efficiency once the number of original features reaches 10. Further, increasing the number of original features yields negligible performance gains. The Balance method (green region) shows improved performance in certain configurations when the number of original features reaches 10 and achieves more excellent stability after reaching 20. At this

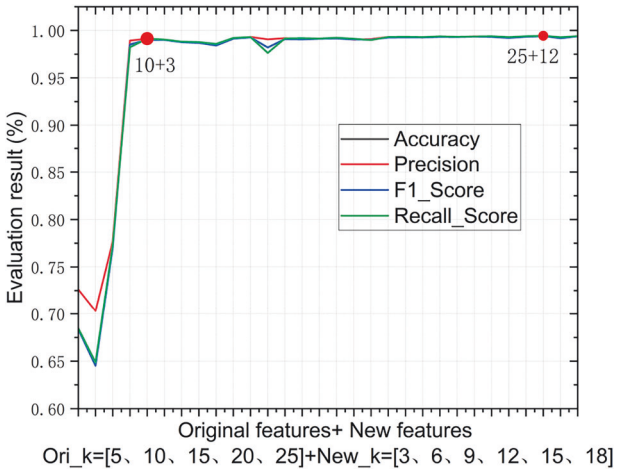


Fig. 9 Performance Evaluation of combined feature space under the Accuracy Strategy: Detailed performance metrics (Accuracy, Precision, F1-score, and Recall) of different (original + new) feature combinations under the Accurate Selection strategy on the InSDN dataset. The x-axis labels denote the number of original features plus the number of added new features

stage, adding more new features has a limited impact on model performance. The Random method (blue region) exhibits significant fluctuation and randomness, with performance less influenced by the number of features. While unpredictable and unstable, it occasionally delivers favorable results.

These results indicate that when the number of features is small, fusing new features can improve model performance more effectively, which is somewhat surprising. From the perspectives of efficiency and stability, the Accuracy-based method achieved the best performance across different feature sizes, followed by the Balance-based method, while the Random-based method performed relatively poorly. Therefore, it is recommended that the Accuracy-based strategy be prioritized in subsequent feature selection for the InSDN dataset.

To further evaluate the performance of the fused features under the Accuracy-based strategy, the results in Fig. 9 indicate that feature fusion significantly improves model performance, but the effects of different fusion combinations vary. When the number of original features is small, the model is more sensitive to the addition of new features. For instance, in the optimal fused matrix space with the "5+x" combination, Accuracy increased from 0.6847 to 0.9914, and the F1-Score improved from 0.6838 to 0.9900, indicating that the new features can effectively compensate for the insufficiency of the original features. As the number of original features increases to 10, 15, 20, and 25, the overall model performance gradually stabilizes, with most metrics remaining around 0.99. Among these, the "25 + 12" combination achieved the following evaluation metrics: Accuracy = 0.9943, Precision = 0.9942, F1-Score = 0.9939, and Recall = 0.9943. Overall, with a relatively sufficient number of original features, a moderate increase in new features can further enhance model performance. However, as the feature scale reaches a certain level, the performance gain gradually diminishes, indicating diminishing marginal returns in feature fusion.

To reduce communication and computational burden while achieving a better performance balance, Section 4.6 ultimately selected 10 original features and fused 3 new features, resulting in a total of 13 features in the optimal fused matrix space. Among the three retained features, one originates from the anomaly-detection algorithm (Isolation Forest) and the other two from the spectral method (Fast Fourier Transform, FFT). The model's evaluation metrics were: Accuracy = 0.9882, Precision = 0.9882, F1-Score = 0.9876, and Recall = 0.9882. The 10 selected original features retain the most discriminative and directly interpretable traffic characteristics. However, the original features alone may not fully capture the hidden relationships and nonlinear structures among traffic variables. The three selected unsupervised features provide complementary latent information by summarizing higher-order relationships that are not explicitly represented by the original variables. When fewer unsupervised features are used, the latent representations may be insufficient to effectively complement the original feature space. Conversely, including more transformed features may introduce additional computational overhead, redundant or weakly relevant information, and increased input dimensionality, while potentially amplifying noise. This not only raises the computational cost but may also reduce generalization performance because of feature redundancy and overfitting. Therefore, combining 10 original features with three unsupervised features provides sufficient complementary information without introducing excessive dimensionality, thereby achieving the best balance among detection accuracy, model complexity, and computational efficiency.

The time analysis in the next section is conducted on the already extracted $\phi_i(X)$. The time for generating is not measured, as the feature extraction heavily depends on the choice and implementation of the underlying unsupervised algorithms. Unsupervised algorithms are typically more computationally intensive than supervised algorithms, and their runtime strongly depends on the choice and implementation of the underlying algorithms. Fusing only 3 new features is also aimed at balancing model performance with computational cost.

4.6 Model performance comparison

For the baseline comparison, we selected representative models from traditional machine learning, deep learning, and ensemble learning families. It is worth clarifying that

Table 3 Hyperparameter settings for deep learning baseline models

Model	Key hyperparameters	Optimizer learning rate	Training settings
Transformer	d_{model} = 192; 4 layers; 4 heads; FFN = 768; dropout = 0.20	AdamW; lr = 2.5×10^{-4}	batch = 1024; epochs = 40; weighted CE
KAN	3 layers; hidden = 256; grids = 8; dropout = 0.10	AdamW; lr = 1×10^{-3}	batch = 1024; epochs = 150; CE
GAT	4 layers; 8 heads; hidden = 256; dropout = 0.10;	AdamW; lr = 1×10^{-3}	batch = 1024; epochs = 150; CE
DNN	Dense = 128/256/128/96; dropout = 0.1	Adam; lr = 1×10^{-4}	batch = 128; epochs = 33; CategoricalCE
DCNN	Conv filters = 128/256/128; Dense=256/128/96/128/96; dropout = 0.1	Adam; lr = 1×10^{-4}	batch = 128; epochs = 33; CategoricalCE
CNN+biLSTM2+DNN	Conv filters = 128/256; BiLSTM hidden = 128; dropout = 0.2	Adam; lr = 5×10^{-4}	batch = 256; epochs = 33; CategoricalCE
CNNBiGRUAttention	channels = 256; BiGRU hidden = 256; 2 layers; dropout = 0.10	AdamW; lr=1×10^{-3}	batch = 1024; epochs = 150; CE

Table 4 Performance evaluation under different models

Model	Accuracy	Precision	F1-Score	Recall	Training time	Inference time
SVM	96.35%	66.51%	64.74%	65.28 %	16.86 s	5.05 s
CatBoost	99.63%	96.59%	95.85%	95.21 %	12.43 min	0.37 s
XGBoost	99.63%	98.37%	98.83%	98.34 %	1.93 min	1.55 s
DNN	98.99%	98.79%	98.99%	98.75 %	4.36 min	3.89 s
DCNN	99.65%	99.68%	99.81%	99.56 %	47.78 min	18.91 s
CNN+biLSTM2+DNN	99.86%	99.86%	99.86%	99.86 %	3.22 h	1.32 min
CNNBiGRUAttention	99.83%	96.19%	88.72%	87.24%	12.11 h	0.59 min
Transformer	99.81%	85.01%	86.46%	93.48%	7.48 h	0.35 min
KAN	99.67%	80.92%	76.53%	73.81%	0.33 h	0.03 min
GAT	93.34%	95.15%	93.94%	93.34%	30.24 h	6.19 min
LightGBM (10 Orig)	98.42 %	98.33 %	98.32%	98.42 %	26.64 s	1.62 s
LightGBM (10 Orig*)	98.42 %	98.33 %	98.32%	98.42 %	8.94–12.55 s	1.59–1.88 s
LightGBM (10 Orig + 3 New)	98.83 %	98.78 %	98.67%	98.83 %	32.16 s	1.61 s
LightGBM (10 Orig + 3 New *)	98.82 %	98.82 %	98.76%	98.82 %	9.65–13.65 s	1.57-1.90 s

* Models trained in a distributed environment with three nodes, all other models were trained on a single machine

XGBOD is not included as a baseline in our evaluation (Table 4), because its original framework was primarily designed for binary anomaly detection, whereas our task is multiclass intrusion detection on the InSDN dataset. Directly applying XGBOD would therefore require non-trivial modifications to its task formulation and classification framework. Moreover, although our feature enhancement strategy is inspired by the general principle of XGBOD, the specific combination of unsupervised models adopted in our framework differs from that used in the original XGBOD. For these reasons, we exclude it to ensure a fair and meaningful comparison. With this in mind, we comprehensively evaluate the performance and computational cost of the selected models on the InSDN dataset. For reproducibility and a fair comparison, the key hyperparameters of the deep learning baseline models are summarized in Table 3. The configurations listed in the table correspond to the actual settings used in the experiments, including the main architecture configurations, optimizer and learning rate, batch size, number of training epochs, and loss function. It can be observed that different models exhibit notable differences in classification performance and computational overhead. Below is a detailed analysis of Table 4.

In terms of classification performance, the conventional machine-learning model, SVM, achieved precision, recall, and F1 scores of approximately 65%, substantially underperforming the other models. This result indicates that SVM has limited capacity to model complex data characteristics. Deep-learning models, by contrast, incurred considerably greater training and inference costs because of their multiple hidden layers and large numbers of trainable parameters. For example, CNN+BiLSTM2+DNN achieved the best predictive performance among the deep-learning models, but required 11,592.675 s—approximately 3.2 h—for

training and 79.052 s for inference. Although models such as GAT and Transformer offer strong representation-learning capabilities, their identification of minority-class samples was comparatively unsatisfactory under imbalanced class distributions. They are therefore more suitable for applications that prioritize predictive performance and have sufficient computational resources. Among the ensemble tree-based models, XGBoost delivered the best overall performance, whereas CatBoost provided the fastest inference and is particularly well suited to the rapid prediction of known attack types.

Focusing further on the ablation study of the proposed framework, we strictly controlled the hyperparameter settings and compared four combinations of single-node versus three-node distributed training and the original 10 features versus the "10 + 3" fused feature space. The results showed that: (1) with the same feature input, switching from single-node to distributed training produced almost no change in predictive performance. The accuracy remained 98.42% for the original feature set and was 98.83% and 98.82% for the fused feature set under single-node and distributed training, respectively. However, the training time was substantially reduced, from 26.64 s to 8.94-12.55 s with the original features and from 32.16 s to 9.65-13.65 s with the fused features. These results indicate that the main contribution of distributed training lies in accelerating large-scale data processing and improving scalability rather than enhancing predictive accuracy. (2) After introducing the three newly constructed fused features, both accuracy and F1-score, as well as the other evaluation metrics, improved under both single-node and distributed settings. This suggests that the complementary information extracted through unsupervised representation learning enhances the feature representation, and that this improvement is independent of the distributed training environment. (3) A comparison of the training time under the original 10-feature configuration and the "10+3" mixed-feature configuration reveals that the use of mixed features increases training overhead. (4) The inference time remained essentially unchanged across the four LightGBM configurations, at approximately 1.6 s, indicating that neither feature enhancement nor distributed deployment adversely affects online inference efficiency.

It should be noted, however, that distributed training incurs communication overhead from per-iteration synchronization of gradients and split-point information. For small-to-medium datasets, this overhead may offset the computational gains; the scalability advantage becomes more evident only with larger data volumes (e.g., millions of samples). In practice, the final choice should therefore weigh hardware configurations, training time, and inference speed requirements. The experimental findings indicate that the improvement in predictive performance was primarily attributable to the inclusion of the learned representation features, which produced consistent gains across all classification metrics. In contrast, the distributed framework primarily reduced training and inference times, thereby improving computational efficiency and scalability. Considering training efficiency, balance across evaluation metrics, and practical deployment requirements, the distributed configuration incorporating representation-learning features achieved a favorable trade-off between predictive performance and computational cost.

From a multi-objective optimization perspective, the compared methods occupy different regions of the performance-efficiency trade-off space.For example, models such as CNN+BiLSTM2+DNN achieve superior predictive performance but incur significantly higher training and inference costs, whereas the proposed distributed LightGBM configuration reduces computational overhead, although the representation learning process introduces additional feature extraction overhead. Therefore, rather than pursuing the absolute optimum of a single metric, the proposed method is positioned in a favorable region of the performance-efficiency trade-off space, making it particularly suitable for resource-constrained SDN environments. In this sense, the model selection problem can be interpreted as a Pareto-type trade-off among predictive performance, training time, inference latency, and resource consumption, where no single approach dominates all others simultaneously.

Overall, no single model dominates the others across both predictive performance and computational efficiency, indicating an inherent Pareto-type trade-off among the compared approaches. The optimal choice is therefore highly application-dependent, necessitating a careful consideration of practical deployment conditions, such as latency tolerance, data scale, and resource availability.

5 Discussion

Overall, this study represents an engineering investigation of intrusion detection in resource-constrained software-defined networking (SDN) environments. It aims to reduce communication overhead, improve computational efficiency, and enhance detection performance. The following discussion examines the proposed approach from four perspectives: performance gains, zero-day attack detection, feature-space diversity, and computational cost.

Discussion 1: Performance Gains. The experimental results indicate that incorporating unsupervised representation learning yields only modest improvements in overall model performance. This finding suggests that the model may be approaching the performance ceiling of the specific datasets used, or that the feature-learning capacity of the base classifier is nearing saturation. In large-scale network

traffic scenarios, even a marginal improvement in the detection rate may provide practical benefits.

Discussion 2: Computational Cost. Improvements in detection performance may also be accompanied by increased computational overhead. In this study, we did not deliberately minimize feature-generation time, because unsupervised algorithms are generally more computationally expensive than supervised methods, and their runtime efficiency largely depends on the choice of the underlying algorithm. Therefore, we combined 10 original features with 3 unsupervised features to provide sufficient complementary information without introducing excessive additional overhead, thereby achieving a favorable balance between detection accuracy and computational efficiency. However, this balance may vary across datasets and application scenarios. Future research could therefore optimize the computational cost of unsupervised feature extraction according to the characteristics of specific data environments, with the aim of further improving overall performance.

Discussion 3: Practical Deployment Considerations. In practical SDN deployments, the proposed framework should be evaluated as part of an end-to-end detection and response pipeline rather than solely on the basis of classifier inference time. The complete process includes traffic collection, flow-level feature extraction, unsupervised feature generation, feature selection, classification, controller decision-making, and flow-table updates. The mechanisms for traffic collection and SDN flow-table operations follow the frameworks described in Jia et al. (2026, 2025). Once suspicious traffic is identified, the detection results can be transmitted to the SDN controller, which can update the corresponding flow rules to block, rate-limit, redirect, or further inspect the identified traffic. Therefore, the feasibility of practical deployment depends on the total latency introduced by data acquisition, feature processing, model execution, and controller response. Although the configuration combining 10 original features with 3 unsupervised features achieves a favorable balance between detection performance and computational efficiency under the current experimental conditions, the actual end-to-end latency may vary depending on network scale, controller workload, hardware resources, and flow-rule update frequency. In addition, the sensitivity of the unsupervised feature-generation algorithms and the LightGBM classifier to hyperparameter settings should be systematically investigated before large-scale deployment. Therefore, future work should separately measure the time required at each processing stage and evaluate the framework under realistic online traffic conditions.

Discussion 4: Comparison with Alternative Paradigms. Federated learning provides an alternative paradigm for privacy-preserving intrusion detection in multi-domain SDN environments. Unlike the centralized architecture considered in this study, federated learning allows multiple SDN domains to collaboratively train a shared model without directly exchanging raw traffic data, thereby reducing concerns related to privacy and data governance. However, this paradigm also introduces additional communication overhead, heterogeneous and non-independent data distributions, model aggregation complexity, and potential poisoning or privacy inference attacks. In contrast, this study focuses on improving feature representation, computational efficiency, and detection performance in resource-constrained centralized SDN environments. Therefore, the two paradigms pursue different but complementary objectives. In future work, the feature-fusion method proposed in this study could be incorporated into a federated learning framework to improve knowledge sharing and model generalization across different SDN domains while preserving the privacy of local traffic data.

6 Conclusion

This study demonstrates three key findings for SDN intrusion detection. First, unsupervised representation learning provides complementary information that modestly improves detection performance even when the original feature space already exhibits strong predictive power. Second, feature selection is essential for balancing performance and computational efficiency—our '10+3' configuration achieves near-optimal detection with 94% feature reduction. Third, distributed LightGBM training effectively reduces training latency without sacrificing accuracy, making the framework suitable for resource-constrained, multi-controller SDN deployments. Future work will explore adaptive feature selection strategies and integration with federated learning for privacy-preserving cross-domain collaboration.

6.1 Nomenclature

The main mathematical symbols used throughout this paper are summarized in Table 5 for quick reference.

Table 5 Summary of key mathematical notations

Symbol	Description
X	Original feature space (n samples × m dimensions)
$\phi(X)$	Feature matrix generated by unsupervised representation learning
FV	Feature vector extracted by a single unsupervised algorithm
k	Number of unsupervised feature extraction algorithms employed
d_i	Dimensionality of the i-th generated feature vector $\phi_i(X)$
S	Selected optimal subset of generated features (p features in total)
p	Number of selected features from the generated space (i.e., $\|S\|$)
CFS	Combined feature space formed by concatenating X and S
C	Number of target classes in the multiclass classification task

Author contributions Yufeng Jia and Jiadong Ren conceptualized and designed the study. Yufeng Jia and Haitao He implemented the model and conducted the primary experiments. Additionally, Yufeng Jia developed the custom Mininet-simulated SDN environment, contributed to the model's mitigation module, and wrote the main manuscript text. Xianshan Li provided critical revisions to enhance the manuscript's quality. Ke Xu prepared the figures. All authors reviewed and approved the final manuscript.

Funding This work was supported by the Natural Science Foundation of Xinjiang Uygur utonomous Region (Grant No. 2025D01C88), S&T Program of Hebei (Grant No. 246Z1817G), and the Research Project of Innovation Capability Improvement Plan Project of Hebei Province (22567626H).

Data availability Publicly available datasets were analyzed in this study. These data can be found here: https://aseados.ucd.ie/datasets/SDN/InSDN_DatasetCSV.zip/. https://cdn.jsdelivr.net/gh/yzhao062/XGBOD@master/datasets/.

Declarations

Conflict of Interest The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

Ali B (2025) On the fog's frontline: a federated machine learning approach for industrial network threat detection and intrusion prevention. J Cybersecur 11(1):041

Alotaibi J (2025) AI-driven intrusion detection and mitigation framework for software-defined IoT networks. Peer-to-Peer Netw Appl 18(6):321

Bhoi U, Goriya N, Nagekar N, Trivedi R, Patel M (2025) AI Intrusion Detection System Using Graph Neural Networks for Software Defined Networks (SDN). Int J Appl Mathematics 38(12s):3189–3210

Chandola V, Banerjee A, Kumar V (2009) Anomaly detection: a survey. ACM Comput Surv 41(3):1–58

Chen Y-F, Lin FY-S, Hsu S-Y, Sun T-L, Huang Y, Hsiao C-H (2025) Adaptive traffic control: Openflow-based prioritization strategies for achieving high quality of service in software-defined networking. IEEE Trans Netw Serv Manag 22(3):2295–2310

Elsayed MS, Le-Khac N-A, Jurcut AD (2020) Insdn: a novel SDN intrusion dataset. IEEE Access 8:165263–165284

Fernandes G, Rodrigues JJPC, Carvalho LF, Al-Muhtadi JF, Proenca Jr, ML (2019) A comprehensive survey on network anomaly detection. Telecomm Syst 70(3):447–489

Jia Y, Ren J, He H, Li X, Zhang P, Li R (2025) Reducing latency in SDN: an innovative closed-loop migration collaboration architecture. Trans Emerg Telecommun Technol 36(3):e70091

Jia Y, Ren J, He H, Li X, Xu K (2026) CBDNN: an efficient and generalized intrusion detection method in SDN environments. Clust Comput 29(4):263

Jiang J-P, Liu S-Y, Cai H-R, Zhou Q-L, Ye H-J (2026) Representation learning for tabular data: A comprehensive survey. IEEE Transac Pattern Anal Mach Intell 48(6):6488–6508

Kaur A, Krishna CR, Patil NV (2025) A comprehensive review on software-defined networking (SDN) and DDoS attacks: Ecosystem, taxonomy, traffic engineering, challenges and research directions. Computer Sci Rev 55:100692

Ke G, Meng Q, Finley T, Wang T, Chen W, Ma W et al (2017) Lightgbm: a highly efficient gradient boosting decision tree. In: Neural Information Processing Systems, NIPS'17, Red Hook, NY, USA. Curran Associates Inc., pp 3149–3157

Li Z, Zhu Y, van Leeuwen M (2024) A survey on explainable anomaly detection. ACM Trans Knowledge Discov Data 18(1):1–54

Nunez-Agurto D, Fuertes W, Marrone L, Benavides-Astudillo E, Vasquez-Bermudez M (2023) Traffic classification in software-defined networking by employing deep learning techniques: A systematic literature review. In: Valencia-García R, Bucaram-Leverone M, Del Cioppo-Morstadt J, Vera-Lucio N, Centanaro-Quiroz PH (eds) Technologies and Innovation. CITI 2023. Communications in Computer and Information Science, vol 1873. Springer, Cham. https://doi.org/10.1007/978-3-031-45682-4_6

Olanrewaju-George B, Pranggono B (2025) Federated learning-based intrusion detection system for the internet of things using unsupervised and supervised deep learning models. Cyber Security Appl 3:100068

Sattarpour S, Barati A, Barati H (2024) Ebids: efficient bert-based intrusion detection system in the network and application layers of IoT. Clust Comput 28(2):138

Xi Y, Tang C, Huang J-J, Hu X, Liu Y, Liu X (2026) Contrastive and dual adversarial representation learning for multi-view clustering. IEEE Trans Knowl Data Eng 38(1):170–182

Xie S, Wu X, Yu Wang M (2025) Semi-patchcore: A novel two-staged method for semi-supervised anomaly detection and localization. IEEE Trans Instrument Measurement 74:1–12

Zhang F, Kuang K, Chen L, You Z, Shen T, Xiao J (2023) Federated unsupervised representation learning. Front Inf Technol Electron Eng 24(8):1181–1193

Zhang J, Lin L, Yang S, Liu J (2026) Self-supervised skeleton-based action representation learning: A benchmark and beyond. Int J Comput Vision 134(1):38

Zhao Y, Hryniewicki MK (2018) Xgbod: Improving supervised outlier detection with unsupervised representation learning. In: 2018 International Joint Conference on Neural Networks (IJCNN), pp 1–8

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.